

5주차 예습과제

☼ 상태	Done
👤 담당자	이서윤(인공지능대학 인공지능데이터사이언스학부)
📅 마감일	10/05/2026
👤 작업 유형	

분류 실습 - 캐글 산탄데르 고객 만족 예측

데이터 전처리

① 데이터 확인

전체 데이터: 76,020개, 371개 컬럼

TARGET: 고객 만족 여부

- 0: 만족 고객 73,012개 (약 96%)
- 1: 불만족 고객 3,008개 (약 4%)

모든 피처는 숫자형 데이터로 구성됨

고객의 만족 여부가 불균형하게 분포하므로 ROC-AUC를 주요 성능 평가 지표로 사용함

② 데이터 정제

var3의 이상치 **999999** 총 116개를 최빈값인 **2**로 대체

고객 식별용 **ID** 컬럼 제거

TARGET을 레이블로 설정하고, 나머지 369개 컬럼을 피처로 사용

③ 데이터 분할

학습 데이터 80%: 60,816개

테스트 데이터 20%: 15,204개

학습 데이터는 조기 중단을 위해 학습용 70%, 검증용 30%로 추가 분할

XGBoost 모델 학습과 하이퍼 파라미터 튜닝

① 기본 모델 학습

n_estimators=500, **learning_rate=0.05** 설정

조기 중단 100회 적용

테스트 데이터로 ROC-AUC 평가

초기 ROC-AUC: 0.8408

② 하이퍼파라미터 튜닝

HyperOpt의 TPE 알고리즘을 이용해 총 50번 탐색하고, 3-Fold 교차 검증으로 최적 조합을 찾음

파라미터	역할	최적값
<code>max_depth</code>	트리의 최대 깊이	6
<code>min_child_weight</code>	자식 노드 분할 제한	2
<code>colsample_bytree</code>	피쳐 샘플링 비율	0.6507
<code>learning_rate</code>	학습률	0.0594

③ 튜닝 결과

튜닝 전 ROC-AUC - 0.8408

튜닝 후 ROC-AUC - 0.8440

하이퍼파라미터 튜닝을 통해 ROC-AUC가 0.0032 향상되었다

LightGBM 모델 학습과 하이퍼 파라미터 튜닝

① 기본 모델 학습

`n_estimators=500` 설정

조기 중단 100회 적용

테스트 데이터로 ROC-AUC 평가

초기 ROC-AUC: 0.8384

② 하이퍼파라미터 튜닝

XGBoost와 동일하게 HyperOpt의 TPE 알고리즘을 이용해 총 50번 탐색하고, 3-Fold 교차 검증으로 최적 조합을 찾음

파라미터	역할	최적값
<code>num_leaves</code>	최대 리프 노드 수	33
<code>max_depth</code>	트리의 최대 깊이	119
<code>min_child_samples</code>	리프 노드의 최소 데이터 수	69
<code>subsample</code>	데이터 샘플링 비율	0.9332

파라미터	역할	최적값
learning_rate	학습률	0.0459

③ 튜닝 결과

튜닝 전 ROC-AUC - 0.8384

튜닝 후 ROC-AUC - 0.8412

하이퍼파라미터 튜닝을 통해 ROC-AUC가 0.0028 향상되었다

Decision-Tree-and-Random-Forest-classifier-models

1. Data Review – 데이터 탐색

① 데이터 확인

데이터셋: drug200.csv

전체 데이터: 200개, 6개 컬럼

결측치 없음

변수	설명
Age	환자의 나이
Sex	성별
BP	혈압 수준
Cholesterol	콜레스테롤 수준
Na_to_K	나트륨/칼륨 비율
Drug	약물 종류 (예측 대상)

② 데이터 분포 확인

약물 종류별 데이터 수

Drug 클래스별 데이터 분포

	count
Drug	
DrugY	91
drugX	54
drugA	23
drugC	16
drugB	16

- DrugY 가 91개로 가장 많고, drugB 와 drugC 가 각각 16개로 가장 적다
- 나이 범위는 15~74세, 평균은 약 44.3세이다
- Age 와 Na_to_K 의 상관계수는 약 -0.063으로 선형 상관관계가 거의 없다

2. Data Visualization – 데이터 시각화

Plotly와 Seaborn을 사용하여 각 변수의 분포와 관계를 시각화

- 막대그래프: 나이, 혈압, 콜레스테롤 분포 확인
- 원형그래프: 성별과 약물 종류별 비율 확인
- 히트맵: 수치형 변수 간 상관관계 확인
- 산점도 및 Pairplot: 나이, Na/K 비율, 약물 종류 간 관계 탐색

3. Data Preparing – 데이터 전처리

① 범주형 데이터 변환

머신러닝 모델에 사용하기 위해 문자열로 표현된 범주형 데이터를 숫자로 변환

변수	변환 방식
Sex	F=1, M=0
BP	HIGH=0, NORMAL=1, LOW=2
Cholesterol	HIGH=1, NORMAL=0
Drug	drugB=0, drugC=1, drugA=2, drugX=3, DrugY=4

변환 후 데이터 타입을 확인하고, 모든 컬럼을 숫자형으로 정리한다

② 데이터 분할

- X (피쳐): Age, Sex, BP, Cholesterol, Na_to_K

- y (레이블): Drug
- 학습 데이터 70%: 140개
- 테스트 데이터 30%: 60개
- `random_state=1` 설정

전처리된 데이터를 활용하여 두 분류 모델을 학습한다

4. Decision Tree Classifier

① 기본 모델 학습

별도의 주요 하이퍼파라미터를 설정하지 않은 기본 Decision Tree 모델을 학습

테스트 정확도: 96.67%

② 분할 기준 비교

트리의 최대 깊이를 3으로 제한하고, 두 가지 분할 기준을 비교

- Gini: 노드의 불순도를 측정하여 데이터를 분할한다
- Entropy: 정보의 불확실성을 측정하여 데이터를 분할한다

평가 항목	Gini	Entropy
<code>max_depth</code>	3	3
학습 정확도	91.43%	91.43%
테스트 정확도	90.00%	90.00%

5. Random Forest Classifier

① 기본 모델 학습

- 기본 Random Forest 모델의 테스트 정확도: 95.00%
- `n_estimators=100` 설정 시에도 동일한 정확도 기록

② 파라미터 탐색

학습 정확도와 테스트 정확도를 비교하며 두 가지 파라미터의 설정값을 변경

파라미터	설명	선택값
<code>random_state</code>	난수 생성 기준	5
<code>n_estimators</code>	생성할 의사결정 트리 수	100

`random_state` 를 0~9까지 변경하고, `n_estimators` 를 10~100까지 비교하여 최종 모델 설정을 선택하였다

③ 최종 모델 결과

최종 설정: `n_estimators=100`, `random_state=5`

학습 정확도 - 100%

테스트 정확도 - 96.67%

기본 모델의 95.00%에서 96.67%로 정확도가 향상되었다

6. Evaluation – 분류 모델 평가

Confusion Matrix (혼동 행렬)

의사결정 트리와 최종 랜덤 포레스트 모델에 대해 혼동 행렬을 생성하고 히트맵으로 시각화함

모델	정확히 분류	오분류
기본 Decision Tree (혼동 행렬용 재학습 모델)	58개	2개
Decision Tree (Gini, 깊이 3)	54개	6개
최종 Random Forest	58개	2개

최종 Random Forest는 테스트 데이터 60개 중 58개를 정확히 예측하였으며, `drugB`에 해당하는 2개 데이터를 `drugA`로 잘못 분류하였다

7. 최종 결과 비교

모델	테스트 정확도
기본 Decision Tree	96.67%
Decision Tree (Gini)	90.00%
Decision Tree (Entropy)	90.00%
기본 Random Forest	95.00%
최종 Random Forest	96.67%

최종적으로 기본 Decision Tree와 최종 Random Forest 모두 96.67%의 테스트 정확도를 기록했다. 여러 트리를 결합하는 Random Forest가 기본 설정 대비 조정 후 성능이 개선되는 것도 확인하였다.

Beginner-friendly-catboost-with-optuna

1. Problem Definition – 문제 정의

① 분석 목표

- 환자의 나이, 성별, 혈압 등 건강 정보를 이용해 심장 질환 유무 예측
- 예측 대상인 `HeartDisease` 는 0과 1로 구분
 - 0: 심장 질환 없음
 - 1: 심장 질환 있음
- 따라서 이진 분류(Binary Classification) 문제에 해당

② 평가 지표

심장 질환 있음 (1) - 508명

심장 질환 없음 (0) - 410명

두 클래스가 비교적 균형 있게 분포하므로 정확도(Accuracy)를 주요 평가 지표로 사용

2. Exploratory Data Analysis – 데이터 탐색

① 데이터 확인

- 전체 데이터: 918개, 12개 컬럼
- 입력 피처: 11개
- 타겟 변수: `HeartDisease`
- 결측치 및 중복 데이터 없음

구분	변수
수치형	Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak
범주형	Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope
타겟	HeartDisease

② 수치형 변수 분석

기술통계량, 히스토그램, 상자그림, 상관관계 히트맵을 이용해 수치형 피처의 분포와 변수 간 관계를 확인

- 각 수치형 변수의 분포 및 왜도 확인
- 전반적으로 타겟과 강한 선형 상관관계는 나타나지 않음
- `Oldpeak` 은 심장 질환 유무와 양의 상관관계를 보임

③ 범주형 변수 분석

범주별 심장 질환 비율을 비교하여 예측에 영향을 줄 수 있는 특성을 탐색함

변수	분석 결과
Sex	남성 63.17%, 여성 25.91%
ChestPainType	ASY 그룹의 심장 질환 비율이 79.03%로 가장 높음
RestingECG	ST 그룹이 65.73%로 가장 높음
ExerciseAngina	Y 그룹 85.18%, N 그룹 35.10%
ST_Slope	Flat 그룹 82.83%, Up 그룹 19.75%

일부 범주형 변수에서는 심장 질환 여부에 따른 뚜렷한 차이가 관찰됨

3. Data Preprocessing – 데이터 전처리

① 데이터 분할

- 전체 데이터 중 학습 데이터 70%, 테스트 데이터 30%로 분할
- CatBoost 최적화를 위해 학습 데이터의 20%를 검증 데이터로 추가 분리
- 테스트 데이터는 최종 평가에 사용

② 범주형 변수 처리

모델의 특성에 따라 서로 다른 전처리 방식을 적용

- One-Hot Encoding: 범주형 데이터를 여러 개의 이진 변수로 변환
- StandardScaler: 일부 모델에서 수치형 데이터의 스케일을 조정
- Pipeline: 전처리와 모델 학습 과정을 하나로 연결
- CatBoost: 별도의 One-Hot Encoding 없이 범주형 변수를 직접 처리

특히 SVM, KNN과 같이 변수의 크기에 영향을 받는 모델에서는 스케일링 전후 성능을 비교하였다

4. Model Selection – 모델 선정 및 성능 비교

① Baseline Model

다른 분류 모델의 성능을 평가하기 위한 기준으로 `DummyClassifier`를 사용

- 모든 데이터를 심장 질환이 있는 클래스(1)로 예측
- 기준 정확도: 59.42%

② 일반 분류 모델

Logistic Regression, Linear Discriminant Analysis, SVM, KNN 모델을 학습하고 스케일링 전후 정확도를 비교함

모델	스케일링 전	스케일링 후
Logistic Regression	88.41%	88.04%
Linear Discriminant	86.96%	86.96%
SVM	72.46%	88.41%
KNN	71.74%	88.41%

SVM과 KNN은 스케일링 이후 정확도가 크게 향상되었지만, Logistic Regression과 Linear Discriminant는 큰 차이를 보이지 않음

③ 앙상블 모델

모델	정확도
AdaBoost	87.32%
Gradient Boosting	87.68%
Random Forest	88.77%
Extra Trees	88.04%

앙상블 모델 중에서는 Random Forest가 가장 높은 정확도를 기록함

④ XGBoost, LightGBM, CatBoost 비교

부스팅 모델별 정확도

최종 노트북의 기본 모델 실행 결과

- XGBoost: 84.78%
- LightGBM: 87.32%
- CatBoost: 89.13%

CatBoost가 세 모델 중 가장 높은 정확도를 기록하였으므로 이후 CatBoost의 하이퍼파라미터를 조정하여 추가적인 성능 개선을 시도함

5. CatBoost Hyperparameter Tuning with Optuna

① Optuna 적용

Optuna는 여러 하이퍼파라미터 조합을 탐색하여 최적의 성능을 내는 설정을 찾는 최적화 라이브러리이다

이번 실습에서는 검증 데이터의 정확도 최대화를 목표로 TPE 방식으로 총 50회 탐색을 수행함

하이퍼파라미터	역할
<code>objective</code>	최적화 목적 함수
<code>colsample_bylevel</code>	트리 분할 시 사용하는 피쳐 비율
<code>depth</code>	트리의 최대 깊이
<code>boosting_type</code>	부스팅 방식
<code>bootstrap_type</code>	데이터 샘플링 방식
<code>subsample</code>	데이터 샘플링 비율 (해당 방식에서 사용 시)
<code>bagging_temperature</code>	베이지안 부트스트랩의 가중치 변동 정도 (해당 방식에서 사용 시)

② 최적 하이퍼파라미터

50회 탐색 결과, 가장 높은 검증 정확도를 기록한 조합은 다음과 같다

파라미터	최적값
<code>objective</code>	CrossEntropy
<code>colsample_bylevel</code>	0.09917
<code>depth</code>	10
<code>boosting_type</code>	Plain
<code>bootstrap_type</code>	MVS

Optuna 최고 검증 정확도 - 92.25%

Optuna가 찾은 최적 조합을 최종 CatBoost 모델에 적용한 후 별도로 분리한 테스트 데이터로 성능을 평가함

③ 튜닝 결과

튜닝 전 테스트 정확도 - 89.13%

튜닝 후 테스트 정확도 - 90.22%

최종 모델은 테스트 데이터 276개 중 약 90.22%를 올바르게 분류함

분류 보고서(Classification Report)의 주요 결과는 다음과 같다

지표	질환 없음 (0)	질환 있음 (1)
Precision	0.88	0.92
Recall	0.88	0.92
F1-score	0.88	0.92

심장 질환이 있는 클래스(1)에서 정밀도, 재현율, F1-score 모두 0.92를 기록함

6. Feature Importance – 피쳐 중요도 분석

최적화된 CatBoost 모델의 피쳐 중요도를 시각화하여 각 변수가 예측에 기여하는 정도를 확인함

- 피쳐별 중요도 값을 추출
- 중요도가 낮은 순서대로 정렬
- 막대그래프를 통해 변수별 상대적 영향력을 비교

이 과정을 통해 모델의 예측 결과에 어떤 변수들이 중요하게 작용했는지 해석할 수 있다

7. Final Model Comparison – 최종 모델 비교

모델	정확도
DummyClassifier	59.42%
Logistic Regression	88.41%
Linear Discriminant	86.96%
SVM (Scaled)	88.41%
KNN (Scaled)	88.41%
AdaBoost	87.32%
Gradient Boosting	87.68%
Random Forest	88.77%
Extra Trees	88.04%
XGBoost	84.78%
LightGBM	87.32%
CatBoost (기본)	89.13%
CatBoost (튜닝 후)	90.22%